Linked List Data Structure

1. Introduction to Linked List

A Linked List is a linear data structure in which elements, called nodes, are stored dynamically and connected using links (pointers or references).

Unlike an array, the elements of a linked list are not required to be stored in contiguous memory locations.

Each node generally contains:

- Data: Stores the value or information.
- Link/Pointer: Stores the reference to another node.

Basic structure of a node

A node contains data and a reference to the next node.

Important terminology

- Head: Reference to the first node.
- Next: Reference to the next node.
- Prev: Reference to the previous node, used in doubly linked lists.
- NULL / None: Indicates the absence of a reference.
- Traversal: Visiting nodes one by one.

2. Types of Linked Lists

There are four important types of linked lists.

1. Singly Linear Linked List

Each node points to the next node. The last node points to NULL.

2. Singly Circular Linked List

Each node points to the next node, and the last node points back to the first node.

3. Doubly Linear Linked List

Each node contains links to both the previous and next nodes. Both ends terminate with NULL.

4. Doubly Circular Linked List

Each node has previous and next links. The last node connects to the first, and the first connects back to the last.

3. Singly Linear Linked List (SLL)

Definition

A Singly Linear Linked List is a collection of nodes in which each node contains data and a reference to the next node. The last node points to NULL.

Diagram

Head → 10 → 20 → 30 → NULL

Node structure in Python

```
class Node:    def __init__(self, data):
                    self.data = data
                    self.next = None
```

Traversal

Traversal starts from head and continues until the reference becomes None.

Insertion operations

1. At the beginning: Create a new node, point it to the current head, and update head.
2. At the end: Traverse to the last node and connect its next to the new node.
3. At a specific position: Reach the preceding node and adjust the links.

Deletion operations

1. From the beginning: Update head = head.next.
2. From the end: Reach the second-last node and set its next = None.
3. By value or position: Find the target node and bypass it by updating the preceding node's link.

Advantages

- Dynamic size.
- Efficient insertion and deletion at the beginning.
- Does not require contiguous memory.
- Requires only one link per node.

Disadvantages

- Supports only forward traversal.
- No direct/random access by index.
- Searching takes linear time.
- Deletion of a node usually requires locating its predecessor.

Applications

- Implementing stacks and queues.
- Polynomial representation.
- Graph adjacency lists.

- Dynamic memory management.

4. Singly Circular Linked List (SCLL)

Definition

A Singly Circular Linked List is a linked list in which each node contains data and a reference to the next node, but the last node points back to the first node instead of NULL.

Key characteristics

- There is no NULL link at the end in a non-empty circular list.
- Traversal can begin at any node.
- The list can be traversed repeatedly in a circular manner.
- A stopping condition is required to avoid an infinite loop.

Advantages

- Suitable for repeated cyclic traversal.
- No need to restart from the head after reaching the last node.
- Useful when implementing circular queues.

Disadvantages

- Traversal termination requires care.
- Incorrect link updates may cause infinite loops.
- Backward traversal is not directly possible.

Applications

- Round-robin CPU scheduling.
- Circular queues.
- Multiplayer turn management.
- Repeating playlists and cyclic task scheduling.

5. Doubly Linear Linked List (DLL)

Definition

A Doubly Linear Linked List is a linked list in which every node contains three fields:

1. prev — reference to the previous node.
2. data — stored information.
3. next — reference to the next node.

The first node's prev is None, and the last node's next is None.

Advantages

- Supports forward and backward traversal.
- Deletion of a known node can be performed in \(O(1)\) time when its links and list membership are available.

s**Disadvantages**

- Requires extra memory for the prev pointer.
- Insertion and deletion require updating more links.
- Incorrect pointer updates can break the list in either direction.

Applications

- Browser forward/backward navigation.
- Undo and redo operations.
- Image viewers and music playlists.
- Deques (double-ended queues).

6. Doubly Circular Linked List (DCLL)

Definition

A Doubly Circular Linked List is a linked list in which each node contains prev, data, and next fields. The last node's next points to the first node, while the first node's prev points to the last node.

Key characteristics

For a non-empty list:

- head.prev points to last node.
- Every node has both prev and next references.
- Traversal is possible in either direction.
- There is no None link between nodes, including at the boundaries.

For a single-node list, insertion and deletion must preserve the circular structure. When the only node is deleted, both head should become None.

Advantages

- Supports forward and backward traversal.
- Allows continuous cyclic traversal in either direction.
- Useful when navigation must wrap around at either end.

Disadvantages

- Requires more memory than singly linked lists.
- More complex pointer management.
- Traversal needs a stopping condition to prevent infinite loops.

Applications

- Music or video playlists with next/previous navigation.
- Multiplayer games and turn management.
- Repeated navigation through records.
- Operating-system process or task scheduling models.

7. Comparison of the Four Types

Feature	**Singly Linear**	**Singly Circular**	**Doubly Linear**	**Doubly Circular**
Pointers per node	1	1	2	2
Forward traversal	Yes	Yes	Yes	Yes
Backward traversal	No	No	Yes	Yes
Last node's next	None	Head	None	Head
Head's prev	Not present	Not present	None	Last Node
Extra memory	Low	Low	Higher	Higher
Traversal termination	None	Back to head	None	Back to starting node
Implementation complexity	Low	Moderate	Moderate	High